extra processing, such as transforming domain-specific data into latent vectors, making it less practical. Despite the limited work in augmenting LLMs with implicit knowledge, researchers are exploring its potential, including its use in storing instructional knowledge about a domain. This approach involves creating an instruction cycle that retrieves the next input prompt from implicit knowledge, parses the LLM's output to recover variable assignments, and stores these back into the memory for retrieving the next instruction. Augmenting LLMs with this instruction cycle allows them to process large inputs and potentially solve complex domain-specific problems [137].

3.1.3 Open Challenges. By incorporating external knowledge, LLMs function like librarians, finding relevant information without needing to memorize all resources. This enhances performance in specialized tasks without extensive retraining, enabling more adaptable and efficient AI systems capable of lifelong learning and knowledge updating. However, augmenting LLMs with external knowledge for domain-specific tasks presents several open challenges.

(1) *Seamless integration*: Seamless integration of external knowledge into LLMs is crucial, whether the knowledge is explicit or implicit. Existing methods typically concatenate retrieved knowledge to the LLM's input or intermediate layers. However, it's important for the LLM to have the option of accepting or rejecting retrieved information, given that such information may be incomplete or conflicting.
(2) *Scalability and adaptability*: Designing systems capable of scaling to manage large amounts of domain-specific data and adapting to new or changing information is challenging. With rapidly expanding knowledge bases, computing pairwise knowledge similarity will become increasingly computationally unfeasible.

3.2 Domain Tool Augmentation

Domain tools refer to specialized software, libraries, or frameworks that are developed specifically for a particular domain or field (*e.g.*, NCBI Web APIs for genomics question answering [64], automated formal theorem prover for mathematical proofs [61], sandbox environment for social behavior simulation [112], etc.). These tools are designed to handle domain-specific tasks, data, or knowledge effectively, which often incorporate algorithms, techniques, or data structures that are tailored to the unique requirements of that domain. However, the utilization of these domain tools often demands strict adherence to input formats or extensive training, making them less accessible to general users. On the other hand, LLMs are artificial general intelligence models that exhibit intelligence and cognitive capabilities across a wide range of tasks and domains. Despite their versatility, current LLMs are constrained in tasks that require domain specialization. These limitations [100, 135] include (1) unstable result formats depending on the random seeds, generation hyperparameters, and input contents [136]; (2) inability to access up-to-date information [108] since LLMs are solely capable of acquiring information from their training data; (3) a tendency to make up facts observed by researchers [59]; (4) lack of precision in certain tasks such as arithmetic [37].

Researchers propose a collaborative integration approach to overcome the limitations of solely using either domain tools or LLMs for complex domain-specific tasks. This approach combines the strengths of both, utilizing domain-specific knowledge, algorithms, and functionalities from the tools, while offering a user-friendly interface through LLMs. This collaboration optimizes the use of domain-specific resources and eases user engagement by allowing LLMs to guide effective tool usage.

LLMs Call Domain Tools. One straightforward way for domain tool augmentation is to allow LLMs to call domain tools. Essentially, this type of approach follows a multi-stage pipeline, given an LLM $f_\Theta(\cdot)$ and a domain tool $\mathcal{T}(\cdot)$: (1) elicit an executable command c for the domain tool from the LLM by curated or constructed prompts p, denoted as

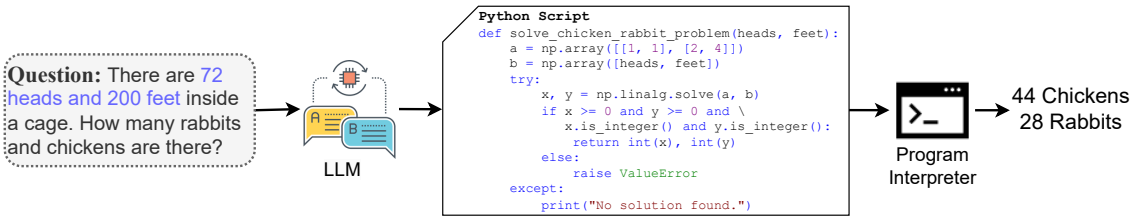

Fig. 3. A toy example for LLMs call domain tools.

$c = f_\Theta(p)$. (2) execute the command c in the domain tool and get the outputs, denoted as $r = \mathcal{T}(c)$. (3) post-process the domain tool outputs by pre-defined rules or the LLM, denoted by $y = post - process(r)$.

This pipeline provides a general diagram and can be easily expanded to multi-LLMs multi-tools collaboration scenarios. The key technical challenge is to ensure the instruction-following and validity of generated commands c from LLMs, so that domain tools can accurately solve desired tasks. Most existing works propose to utilize zero-show or few-shot prompting for executable commands generation (please refer to Sec. 4 for more details). Figure 3 shows a toy example, where the task is to solve an arithmetic question "*There are 72 heads and 200 feet inside a cage. How many rabbits and chickens are there?*". To elicit LLMs to generate an executable Python program, we can formulate the prompt as "*Please write a Python script to solve the arithmetic question. Question: {question_text}*". Then, a snippet of scripts is returned by LLM as the executable command c for the Python interpreter. Finally, the Python interpreter responds with the program outputs "*44, 28*", and further post-processed into desired results "*44 Chickens and 28 Rabbits*".

Depending on the types of domain tools, LLMs can generate corresponding commands that adhere to the syntax and format requirements to call them. Many domain tools provide APIs for easy and precise access. Early exploration in this direction is to elicit elicit search engine queries (*e.g.*, WebGPT [108], WizInternet [69], GopherCite [97]) written in natural language or database queries (*e.g.*, Binder-SQL [19], DIN-SQL [118], BIRD [78]) in the programming language. Later, researchers study how to elicit LLMs to write executable codes that can be executed in program interpreters such as Python [17, 37, 148], Wolfram [168], and so on. Other than the widely-used search queries, database queries, and Python programs, there exist many domain-specialized APIs that have unique syntax. For instance, chatGPT plugin system [109] introduces how to utilize tools for travel booking, restaurant reservation, e-commerce shopping, and workflow automation. These API calling scripts are typically generated by zero-shot or few-shot prompting techniques, as stated in the toy example.

Some complex tasks may involve more than one type of tool to accomplish. Following this vibe, researchers start to generalize LLMs as **task planners** (also mentioned as "API selectors" or "controllers") that call multiple types of domain tools. Other than generating executable commands for each used tool, these approaches focus on how to decompose a complex task into a set of concrete subtasks and how to coordinate between multiple tools. For instance, DSP [61] proposes a Draft, Sketch, and Prove framework for automated theorem proofs where (1) an LLM or oracle is used to draft informal proofs described in a mixture of natural and mathematical languages from input statements, (2) another LLM is used to generate formal sketch from previous informal proof, and (3) off-the-shelf prover is used to prove the open conjectures inside each formal sketch. TaskMatrix.AI [81] proposes using LLMs to derive high-level solution outlines for domain-specific tasks, and automatically match some of the sub-tasks in the outlines to the off-the-shelf domain models/systems to complete them. HuggingGPT [140] proposes leveraging LLMs to act as the controllers to manage existing domain models to solve complicated domain tasks. Qin et al. [122] propose a general tool-augmented LLMs

framework to decompose complex tasks into several subtasks, dynamically adjust the execution plan, and effectively finish each subtask with appropriate tools.

LLMs Embodied to Domain Tools. LLMs can also be called by domain tools to serve as smart agents in interactive environments, namely *LLMs embodied to domain tools*. LLMs, when embodied in interactive robots, can serve as the decision-making module for domain-specific applications. For example, ProgPrompt [144] investigates LLMs' ability to assist robots in completing tasks, when the robot's perception module observes surrounding objects and the LLM is prompted with available action specifications. Results indicate the LLM can generate situated actions for simulated household and real-world tabletop tasks. Furthermore, Murali et al. [107] employ LLMs as the primary component for identifying different speakers in multiparty conversations involving a social robot. The robotics community is progressively exploring these areas, studying LLM utility in human-robot interfaces, planning, grounding, and more [27, 80, 164]. Furthermore, researchers start to investigate how multiple LLMs can *interact with the environment* or *communicate and collaborate* together for real-world task-solving. Mind's eye [84] studies how LLMs can benefit from the interaction with simulated physics engines to inject grounded rationale for physics alignment tasks. CAMEL [76] proposes a communicative agent framework to assign different roles to LLM agents so that multiple AI agents can collaboratively communicate by chatting with each other in an instruction-following fashion to solve the specified task. A recent work [112] utilizes twenty-five LLMs as generative agents in a game-based sandbox environment to create believable simulations of human behavior for interactive applications.

3.2.1 Open Challenges. By leveraging the power of LLMs, domain tools can assist in a variety of tasks across multiple fields, including robotics, virtual agents, and problem-solving in real-world scenarios. This allows for more intuitive and seamless human-machine collaboration, leading to increased efficiency and adaptability in tackling complex problems.

Augmenting LLMs with domain tools poses several open challenges:

(1) *Automated integration*: At present, augmenting LLMs with domain-specific tools requires a significant amount of effort to ensure proper integration. A promising future direction involves utilizing LLMs as a unified interface through standardized protocols to connect various applications and services, thereby enabling seamless communication and interaction between them.
(2) *Getting rid of domain tools*: Another direction for the future development of LLMs is to focus on creating a powerful artificial general intelligence (AGI) model that is not dependent on external tools or domain-specific knowledge. An AGI model would have the potential to revolutionize the way we use language models, enabling more complex and sophisticated tasks to be performed with greater ease and efficiency.

4 PROMPT CRAFTING FOR DOMAIN SPECIALIZATION

While LLMs trained on large-scale corpora are powerful, further pre-training on prompts can enhance their ability to adhere to user intentions and generate accurate and less toxic responses [111, 161]. Prompts, or task-specific input texts designed to elicit specific model responses, help guide the LLM's content generation process and set expectations for the desired output. Approaches generally fall into two categories: (1) **Discrete Prompt** involves creating task-specific natural language instructions to prompt LLMs, eliciting domain-specific knowledge from their parameter space, and (2) **Continuous Prompt** uses learnable vectors to prompt LLMs, eliminating the need for manually designed text instructions. This section delves into both approaches and the merits and limitations of domain specialization.